Group E.C. Homework 2 - NOSQL mini project using document and graph database

A Movie Recommendation System Using MongoDB and Neo4j

Group Members :
Shubhi Sehrawat (020761780)
Shaili Girishkumar Pandya (020744529)
Jyoti Priya (020092722)
Sanjana Dodda (020758985)

Abstract

The project focuses on building an explainable movie analytics and recommendation system based on the MovieLens ml-latest-small database. Using the same database, we will process it using two NoSQL databases, i.e., MongoDB and Neo4j. MongoDB will contain flexible documents of movies, ratings, and tags and will allow us to perform analyses through aggregations. On the other hand, Neo4j will allow modeling user-movie, genre-ratings, and tag-user graphs and making recommendations through relationship chains. The project aims to show that document and graph databases can solve different parts of the same multidisciplinary problem.

1. Introduction and Motivation

The online movie websites need to process huge amounts of user activities in order to categorize and give relevant recommendations. The recommendation becomes more relevant if it can have some evidence behind it, like a user's previous ratings, user preferences, similarity, genre matching, and common tags. This research project aims to examine how various NoSQL database models can help to perform the given task.

In the research project, MongoDB will be used to perform document-based analytics and Neo4j will be used to perform graph discovery. Instead of using these two databases interchangeably, this project tries to find out which of the databases is more suitable for performing each kind of queries.

2. Problem Statement

The problem is designing and implementing an analysis of movies that can recognize the best movies, analyze genres and tags activities, and suggest new movies that have not been rated yet by a user. This analysis needs to be explained, meaning that the movie recommendation is supposed to be based on the evidence that this movie is backed by users sharing high-rated movies with the target one.

3. Project Objectives

- Load and pre-process the MovieLens CSV files using Python and Pandas.
- Utilize MongoDB collections and aggregation pipelines for insights on movies, genres, ratings, and tags.
- Implement Neo4j nodes and relationships to represent users, movies, genres, and rating activity.
- Develop a collaborative recommendation Cypher query.
- Contrast the document, graph, and relational database paradigms.
- Provide reproducible output, visuals, sources, and code implementations.

4. Dataset Description

The project uses the MovieLens ml-latest-small dataset published by GroupLens. The dataset contains 9,742 movies, 100,836 ratings, 3,683 tag applications, and 610 anonymized users. The data were generated from MovieLens activity and are intended for development and experimentation. No user demographic information is included.

File	Important fields	Use in this project
movies.csv	movieId, title, genres	Movie metadata and genre analysis
ratings.csv	userId, movieId, rating, timestamp	Movie ranking and recommendations
tags.csv	userId, movieId, tag, timestamp	Tag frequency and movie description analysis
links.csv	movieId, imdbId, tmdbId	External identifiers and future enrichment

5. System Architecture and Workflow

The process follows an ETL cycle. First, Python scans the source data, validates its schema, transforms the genre column, and generates records for further processing. Then, the validated records are stored in MongoDB and Neo4j databases. Finally, statistics are obtained via aggregations in MongoDB and traversals using Cypher queries. The results are exported as CSV files, graphs, and report artifacts.

Stage	Activity	Technology
Extract	Read movies, ratings, tags, and links CSV files	Python and pandas
Transform	Split pipe-separated genres, preserve identifiers, calculate summaries	Python and pandas
Load	Insert documents into collections and merge graph nodes and relationships	PyMongo and Neo4j Driver
Analyze	Run aggregation pipelines and graph recommendation queries	MongoDB and Cypher
Present	Export tables and create a genre visualization	CSV and matplotlib

6. Data Modeling

6.1 MongoDB document model

MongoDB stores the data in dynamic collections that go by the name of movies, ratings, and tags. The ratings collection has one document per rating done by the users. It is ideal for aggregation and averaging. The movies collection has the title of the movie and the original genre field. The tags collection has free-form data entered by the users without any defined schema.

```
{
 "movieId": 1,
 "title": "Toy Story (1995)",
 "genres": "Adventure|Animation|Children|Comedy|Fantasy"
}
```

6.2 Neo4j graph model

The recommendation is modeled as a graph using Neo4j. The User and Movie nodes have a relationship named RATED which consists of rating and timestamp information. The Movie node is linked with the Genre node using the IN_GENRE relationship. This allows one to trace a path from a target User to movies he/she liked, to other Users liking similar movies and finally candidate movies rated highly by such Users.

Graph element	Meaning
User	An anonymized MovieLens user
Movie	A movie with a unique movieId and title
Genre	A genre derived from the pipe-separated source field
RATED	A user rating with rating and timestamp properties
IN_GENRE	A movie-to-genre relationship

7. MongoDB Implementation and Analysis

MongoDB loader deletes the records from the database that belong to this project, then adds the movies, ratings, and tags and builds indexes on movieId and userId within the ratings collection. It is done in order to facilitate the search process in the future. Aggregation works through grouping the ratings according to movies, counting them, calculating their averages, deleting movies that have less than 50 ratings, sorting and joining them to movie titles with $lookup.

```
db.ratings.aggregate([
  {$group: {_id: "$movieId", ratingCount: {$sum: 1},
        averageRating: {$avg: "$rating"}}},
  {$match: {ratingCount: {$gte: 50}}},
  {$sort: {averageRating: -1, ratingCount: -1}},
  {$limit: 10}
])
```

Minimum Threshold Rating: It is an essential factor because a film with one exceptionally high rating should not be placed higher than a film with a hundred ratings. MongoDB analysis further determines the number of films in each genre, average ratings, and popular tags.

8. Neo4j Implementation and Recommendation Query

A uniqueness constraint on the identifier for Movie and User nodes is set before loading data using the Neo4j loader. Then merging is done of movie nodes, user nodes, RATED relationship, and GENRE relationship. The recommendation process is carried out using a simple neighborhood technique of collaborative filtering. This means that for a chosen user, movies that have been rated above 4.0 are selected, users that have rated such movies are found, and other movies that they have rated are found.

```
MATCH (u:User {userId: 1})-[r1:RATED]->(liked:Movie)
    <-[r2:RATED]-(other:User)
    -[r3:RATED]->(candidate:Movie)
WHERE r1.rating >= 4.0
  AND r2.rating >= 4.0
  AND r3.rating >= 4.0
  AND NOT (u)-[:RATED]->(candidate)
WITH candidate, count(DISTINCT other) AS sharedUsers,
   avg(r3.rating) AS predictedRating
RETURN candidate.title AS recommendation,
     sharedUsers, predictedRating
ORDER BY sharedUsers DESC, predictedRating DESC
LIMIT 10;
```

This recommendation is explainable through the sharedUsers variable, which denotes the number of supporting users who have provided evidence for the candidate. The graph path could also be shown in Neo4j Browser as visual evidence of the traversal process.

9. Results

Upon setting the minimum threshold of 50 ratings, the top rated movies yield the following output. The rankings are determined by average rating and number of ratings. The information is obtained using the provided MovieLens dataset.

Movie	Average rating	Ratings
Shawshank Redemption, The (1994)	4.43	317
Godfather, The (1972)	4.29	192
Fight Club (1999)	4.27	218
Cool Hand Luke (1967)	4.27	57
Dr. Strangelove or: How I Learned to Stop Worrying and Lov	4.27	97
Rear Window (1954)	4.26	84

Godfather: Part II, The (1974)	4.26	129
Departed, The (2006)	4.25	107
Goodfellas (1990)	4.25	126
Casablanca (1942)	4.24	100

From the above, it can be observed that Drama and Comedy genres have the highest number of movies in the dataset, followed by Thriller and Action genres. This is the distribution of the movie records and not a reflection of preferences of the audience. Genres based on rating analysis are carried out separately.

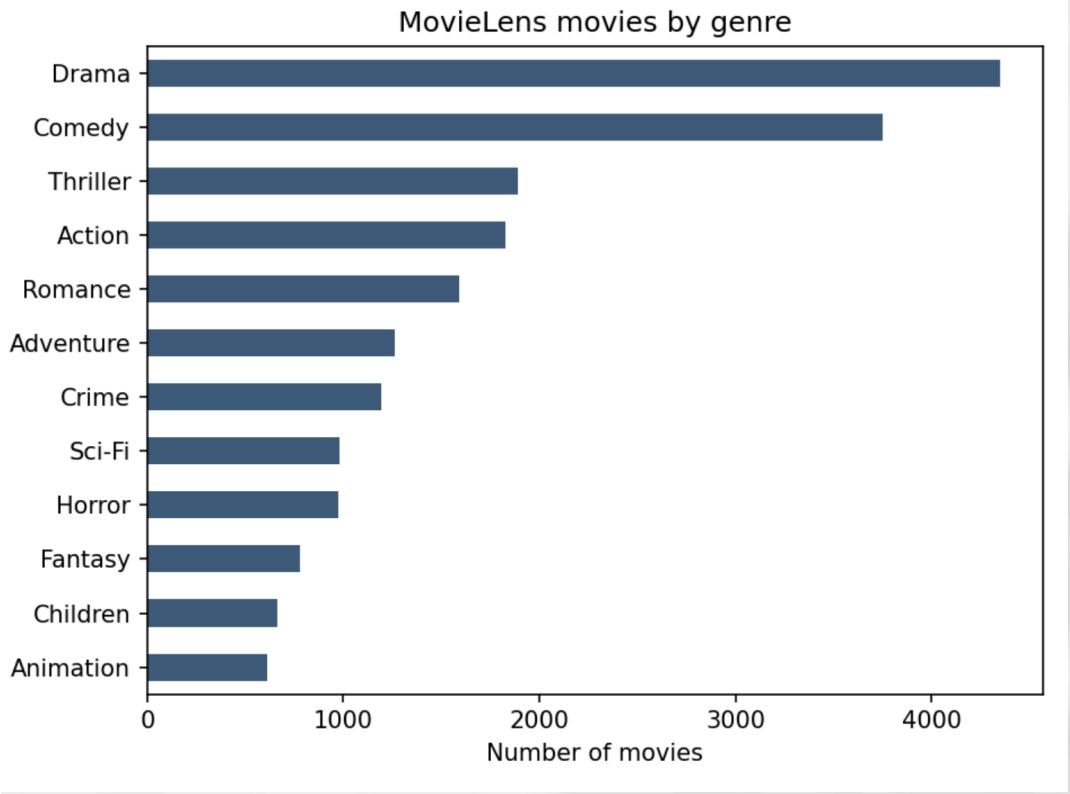


Figure 1. Distribution of movies across the most common genres.

The tag output also provides useful descriptive information. For example, tags such as In Netflix queue, atmospheric, superhero, thought-provoking, Disney, funny, and surreal appear frequently in the supplied tag activity. These tags can support future content-based recommendations or graph relationships.

10. Comparison of MongoDB, Neo4j, and RDBMS

Criterion	MongoDB	Neo4j	RDBMS comparison
Data model	Flexible documents	Nodes and relationships	Normalized tables
Best use in this project	Aggregations and summaries	Multi-hop recommendations	Structured joins and integrity
Main query style	Aggregation pipeline	Cypher traversal	SQL joins and grouping
Strength	Fast to load semi-structured records	Makes relationships explicit	Strong consistency and constraints
Limitation	Complex relationships require more lookup logic	Requires deliberate graph modeling	Many-to-many recommendations require join tables

It is easier to load the data into MongoDB and calculate averages and counts. It is easier to write the recommendation query using Neo4j since the recommendation query needs relationships between users and

movies in succession. RDBMS would have good integrity and SQL functionality, but the recommendation query needs multiple join tables. The best fit depends on the pattern that is used more often, and not because one database is better than another.

11. Technical Difficulties and Key Learnings

Since the source genres are concatenated into a single string separated by pipes, they need to be transformed before they can be processed individually or used as relationships in the graph. An additional problem to be addressed is the possible erroneous sorting based on movies with very low counts of votes. To this end, a minimum count of ratings will be applied. Additionally, Neo4j requires unique constraints before MERGE operations to prevent duplication of nodes during multiple loads.

- Always preserve the movieId and userId in all transformations.
- Create indexes in MongoDB for fields used in group and lookup stages.
- Create uniqueness constraints in Neo4j before creating graph entities.
- Use a minimum rating threshold in comparison of movie average.
- Do not include the downloaded dataset in the code submission package.
- Employ Docker Compose for reproducibility of local database setup.
- Do not generalize development-dataset results as universally valid conclusions.

12. Limitations and Future Work

Recommendation query is a straightforward neighborhood-based algorithm, and it is not a full-fledged production recommendation system. The query does not assess any of the aspects such as precision, recall, ranking, or cold start capabilities. The algorithm also does not implement any kind of time decay; therefore, all old ratings are considered equally influential as more recent ones. There are no demographics of users and no context of viewing in this dataset. Finally, the present implementation processes tags stored in MongoDB, but it does not create Tag nodes in Neo4j.

13. CLO and Rubric Alignment

Requirement	Evidence in the project
CLO1: Understand NoSQL, warehouses, pipelines, and big data technologies	Explains document and graph NoSQL models, ETL workflow, indexing, constraints, Docker, and scale-out extensions.
CLO2: Choose appropriate technologies	Uses MongoDB for flexible aggregation and Neo4j for relationship-based recommendation traversal, with an explicit comparison to RDBMS.
CLO6: Create and solve a multidisciplinary problem	Builds an explainable movie recommendation problem combining data cleaning, databases, graph analysis, programming, and visualization.
Meaningful goal	Produces movie analytics and recommendations that can be explained through ratings and shared user preferences.
Technical difficulty	Uses Python, pandas, PyMongo, the Neo4j driver, JavaScript aggregation, Cypher, Docker, indexes, constraints, and visualization.
Evidence and reproducibility	Includes source links, scripts, CSV outputs, a chart, database queries, and a repeatable local workflow.

14. Conclusion

This project proves that MongoDB and Neo4j can be seen as complementary technologies to store movie data. MongoDB is good at storing flexible records and producing statistical analysis via the process of aggregation.

Neo4j is good at modeling and analyzing relationships used for explanations of recommendations. Together, they achieve the aim of the project in converting the same dataset into two practical NoSQL schemas with explanations.

15. Dataset and Technical Sources

- MovieLens dataset download: https://files.grouplens.org/datasets/movielens/ml-latest-small.zip
- MovieLens datasets page: https://grouplens.org/datasets/movielens/
- Harper and Konstan, The MovieLens Datasets: History and Context: https://doi.org/10.1145/2827872
- MongoDB aggregation pipeline documentation: https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
- Neo4j Cypher manual: https://neo4j.com/docs/cypher-manual/current/
- Neo4j Python driver manual: https://neo4j.com/docs/python-manual/current/

Did you find or come across solutions to similar problems by using Generative AI or other sources?

Yes. We referred to Generative AI while creating the project for understanding different aspects of MongoDB document schema design, aggregation pipeline stages, Neo4j nodes & relationships schema, Cypher query, ETL workflow, report schema etc. We also referred to the official documentation of MovieLens, MongoDB, Neo4j provided in the sources section. However, we did not take a whole solution from some other project. The idea of the project, data-loading process, queries to execute, numerical outputs, visualization, interpretations and limitations were analyzed and validated by our group on the basis of MovieLens data set. We took the help of Generative AI for understanding different aspects of technical solutions but we created our own solution for our own data set.